```
import zipfile

with zipfile.ZipFile('UCI_Credit_Card.csv.zip', 'r') as zip_ref:
    zip_ref.extractall()
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score, roc_curve
```

```
df = pd.read_csv("UCI_Credit_Card.csv")

df.head()
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	P
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	28959.0	29547.0	2000.0	
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	19146.0	19131.0	2000.0	

5 rows × 25 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  float64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  float64
 13  BILL_AMT2                   30000 non-null  float64
 14  BILL_AMT3                   30000 non-null  float64
 15  BILL_AMT4                   30000 non-null  float64
 16  BILL_AMT5                   30000 non-null  float64
 17  BILL_AMT6                   30000 non-null  float64
 18  PAY_AMT1                    30000 non-null  float64
 19  PAY_AMT2                    30000 non-null  float64
 20  PAY_AMT3                    30000 non-null  float64
 21  PAY_AMT4                    30000 non-null  float64
 22  PAY_AMT5                    30000 non-null  float64
 23  PAY_AMT6                    30000 non-null  float64
 24  default.payment.next.month  30000 non-null  int64
dtypes: float64(13), int64(12)
memory usage: 5.7 MB
```

```
df = df.drop('ID', axis=1)
```

```
df['default.payment.next.month']
```

	default.payment.next.month
0	1
1	1
2	0
3	0
4	0
...	...
29995	0
29996	0
29997	1
29998	1
29999	1

30000 rows × 1 columns

dtype: int64

```
X = df.drop('default.payment.next.month', axis=1)
y = df['default.payment.next.month']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Precision:", precision_score(y_test, y_pred_dt))
print("Recall:", recall_score(y_test, y_pred_dt))
print("ROC-AUC:", roc_auc_score(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.7255
Precision: 0.38105413105413105
Recall: 0.4074638233054075
ROC-AUC: 0.6110286899757249
```

```
ada = AdaBoostClassifier(n_estimators=100, random_state=42)

ada.fit(X_train, y_train)

y_pred_ada = ada.predict(X_test)
```

```
print("AdaBoost Accuracy:", accuracy_score(y_test, y_pred_ada))
print("Precision:", precision_score(y_test, y_pred_ada))
print("Recall:", recall_score(y_test, y_pred_ada))
print("ROC-AUC:", roc_auc_score(y_test, y_pred_ada))
```

```
AdaBoost Accuracy: 0.8181666666666667
Precision: 0.6801948051948052
Recall: 0.31911652703731913
ROC-AUC: 0.6385426885239934
```

```
rf50 = RandomForestClassifier(n_estimators=50, max_depth=10, random_state=42)

rf50.fit(X_train, y_train)

y_pred_rf50 = rf50.predict(X_test)
```

```
rf100 = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)

rf100.fit(X_train, y_train)

y_pred_rf100 = rf100.predict(X_test)
```

```
rf200 = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
```

```
rf200.fit(X_train, y_train)

y_pred_rf200 = rf200.predict(X_test)
```

```
results = pd.DataFrame({
    "Model": ["Decision Tree", "AdaBoost", "RF-50", "RF-100", "RF-200"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_ada),
        accuracy_score(y_test, y_pred_rf50),
        accuracy_score(y_test, y_pred_rf100),
        accuracy_score(y_test, y_pred_rf200)
    ]
})

results
```

```
results.plot(x="Model", y="Accuracy", kind="bar")

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_161/2647703678.py in <cell line: 0>()
----> 1 results.plot(x="Model", y="Accuracy", kind="bar")
      2
      3 plt.title("Model Accuracy Comparison")
      4 plt.ylabel("Accuracy")
      5 plt.show()

NameError: name 'results' is not defined
```

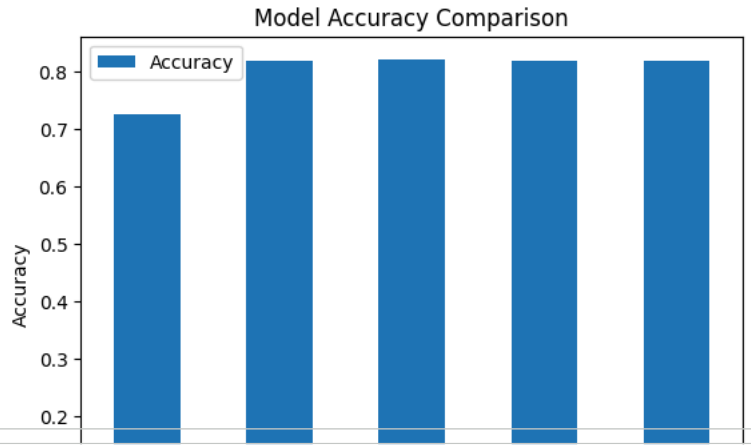
Next steps: Explain error

```
results = pd.DataFrame({
    "Model": ["Decision Tree", "AdaBoost", "RF-50", "RF-100", "RF-200"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_ada),
        accuracy_score(y_test, y_pred_rf50),
        accuracy_score(y_test, y_pred_rf100),
        accuracy_score(y_test, y_pred_rf200)
    ]
})

results
```

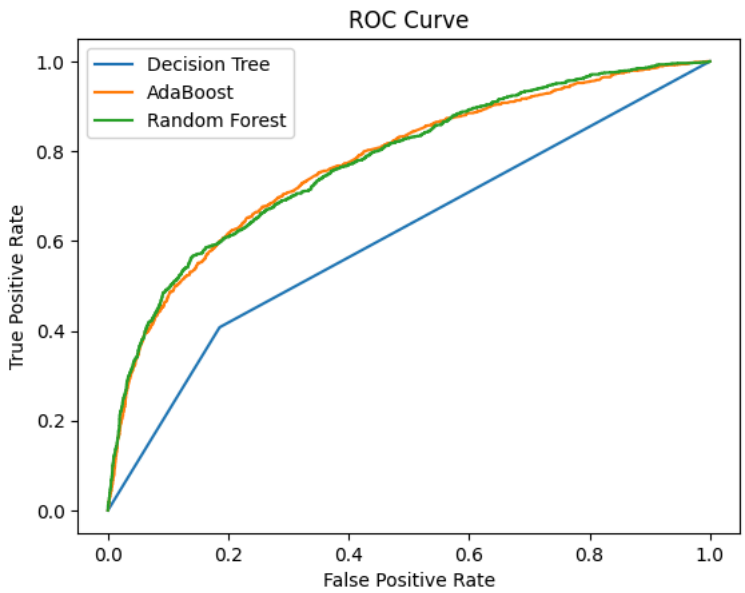
	Model	Accuracy
0	Decision Tree	0.725500
1	AdaBoost	0.818167
2	RF-50	0.820667
3	RF-100	0.819333
4	RF-200	0.819000

Next steps: Generate code with results New interactive sheet

```
results.plot(x="Model", y="Accuracy", kind="bar")

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```




```
fpr, tpr, _ = roc_curve(y_test, dt.predict_proba(X_test)[:,1])
plt.plot(fpr, tpr, label="Decision Tree")

fpr, tpr, _ = roc_curve(y_test, ada.predict_proba(X_test)[:,1])
plt.plot(fpr, tpr, label="AdaBoost")

fpr, tpr, _ = roc_curve(y_test, rf100.predict_proba(X_test)[:,1])
plt.plot(fpr, tpr, label="Random Forest")

plt.legend()
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
```



Random Forest with higher n_estimators performed best because multiple trees reduce variance and improve prediction stability. AdaBoost improved performance compared to a single Decision Tree but Random Forest achieved the highest accuracy and ROC-AUC score.